Informe técnico — RED-LION-INTERCONNECTION

Nov 11, 2026 · @angel

Datos del trabajo

Campo	Detalle
Institución	Fundación Universitaria Compensar
Programa	Ingeniería en Telecomunicaciones
Asignatura	Conmutación y Teletráfico
Docente	Juan Carlos Arbelaez
Autor	Angel Mauricio Cruz Gonzalez
Entregables	Cortes 2 y 3
Fecha	11 de noviembre de 2026
Repositorio público	github.com/angelmacruz123/red-lion-interconnection

Resumen

RED-LION-INTERCONNECTION es una aplicación web en Python y Streamlit que automatiza el ciclo completo de una red corporativa: requerimientos, direccionamiento IP, topología, configuración multi-marca y despliegue en los equipos reales. Un cambio en los requerimientos se propaga solo a todos los módulos.

En el escenario de referencia (edificio de 15 pisos, 600 hosts, red 10.20.0.0/16) la app genera 21 VLAN, 37 nodos y 34 archivos de configuración para Cisco IOS-XE, Huawei VRP, Fortinet FortiOS y MikroTik RouterOS. Se conecta a los equipos por cable de consola serial, SSH o Telnet de laboratorio y aplica nombre, VLAN, IP y puertos con respaldo previo, detención ante errores y verificación final.

La batería automatizada ejecuta 218 pruebas, todas superadas, y 9 controles de seguridad cumplidos, lo que da un porcentaje de confianza del 100 %.

Palabras clave: VLSM, IPv6, NetDevOps, automatización de red, SSH, consola serial, Streamlit, ciberdefensa.

1. Introducción y objetivos

Diseñar la red de un edificio corporativo exige calcular decenas de subredes, documentar la topología y escribir cientos de líneas de configuración por equipo. Hecho a mano, cada cambio de requerimientos obliga a rehacer todo y abre la puerta a errores de direccionamiento y de sintaxis.

Este trabajo resuelve ese problema con una sola herramienta que toma los requerimientos como única fuente de verdad y genera, de forma determinista, el plan IP, el diagrama, las configuraciones y su aplicación en los equipos.

Objetivo general

Desarrollar una aplicación web que modele, documente, configure y verifique la infraestructura de red de un escenario corporativo multi-piso o multi-sede, con conexión real a los equipos.

Objetivos específicos

1. Parametrizar el escenario (pisos, hosts, servicios, VPN, redes base) en un estado global compartido por todos los módulos.
2. Calcular un plan IPv4 con VLSM, enlaces WAN /30 y bloques IPv6 /64 y /127 usando la librería estándar `ipaddress`.
3. Generar la topología jerárquica directamente desde el plan calculado.
4. Producir configuraciones listas para Cisco, Huawei, Fortinet y MikroTik.
5. Conectarse a los equipos por consola serial, SSH o Telnet y aplicar la configuración de forma segura.
6. Medir la calidad del sistema con pruebas automatizadas y un porcentaje de confianza.

2. Requerimientos y alcance

La aplicación cubre seis módulos obligatorios; cada requerimiento funcional se cumple en un módulo y se verifica con pruebas automáticas.

Requerimientos funcionales

ID	Requerimiento	Módulo
RF-01	Configurar escenario: nombre, pisos/zonas, hosts, presupuesto, ancho de banda ISP, servicios, VPN, redes base IPv4/IPv6	M1
RF-02	Sincronizar el diseño en un estado global que recalcula todos los módulos	M1
RF-03	Calcular subredes por piso con VLSM: red, máscara, gateway, rango útil, broadcast	M2
RF-04	Asignar un /64 IPv6 por VLAN y enlaces punto a punto /30 (IPv4) y /127 (IPv6)	M2
RF-05	Dibujar la topología jerárquica desde el plan calculado y exportarla	M3
RF-06	Generar configuraciones para Cisco, Huawei, Fortinet y MikroTik y descargarlas en .txt	M4
RF-07	Conectarse por consola serial, SSH o Telnet de laboratorio y ejecutar comandos en lote	M5
RF-08	Aplicar en el equipo nombre, VLAN, IP y puertos siguiendo el plan, con respaldo y verificación	M5
RF-09	Validar entradas, bloquear inyección de comandos y evitar fuga de credenciales	M6
RF-10	Ejecutar pruebas y calcular el porcentaje de confianza en `docs/pruebas.md`	M6

Requerimientos no funcionales

- **Escalabilidad:** de 1 a 60 pisos/zonas y hasta 20.000 hosts.
- **Exactitud:** todo cálculo IP usa `ipaddress`; nada se inventa.
- **Seguridad:** credenciales solo en `.env`; secretos como marcadores en los archivos generados.
- **Usabilidad:** interfaz responsiva, tema oscuro con acentos rojos, en español.
- **Mantenibilidad:** núcleo separado de la interfaz; un fabricante nuevo es un archivo.

Fuera de alcance

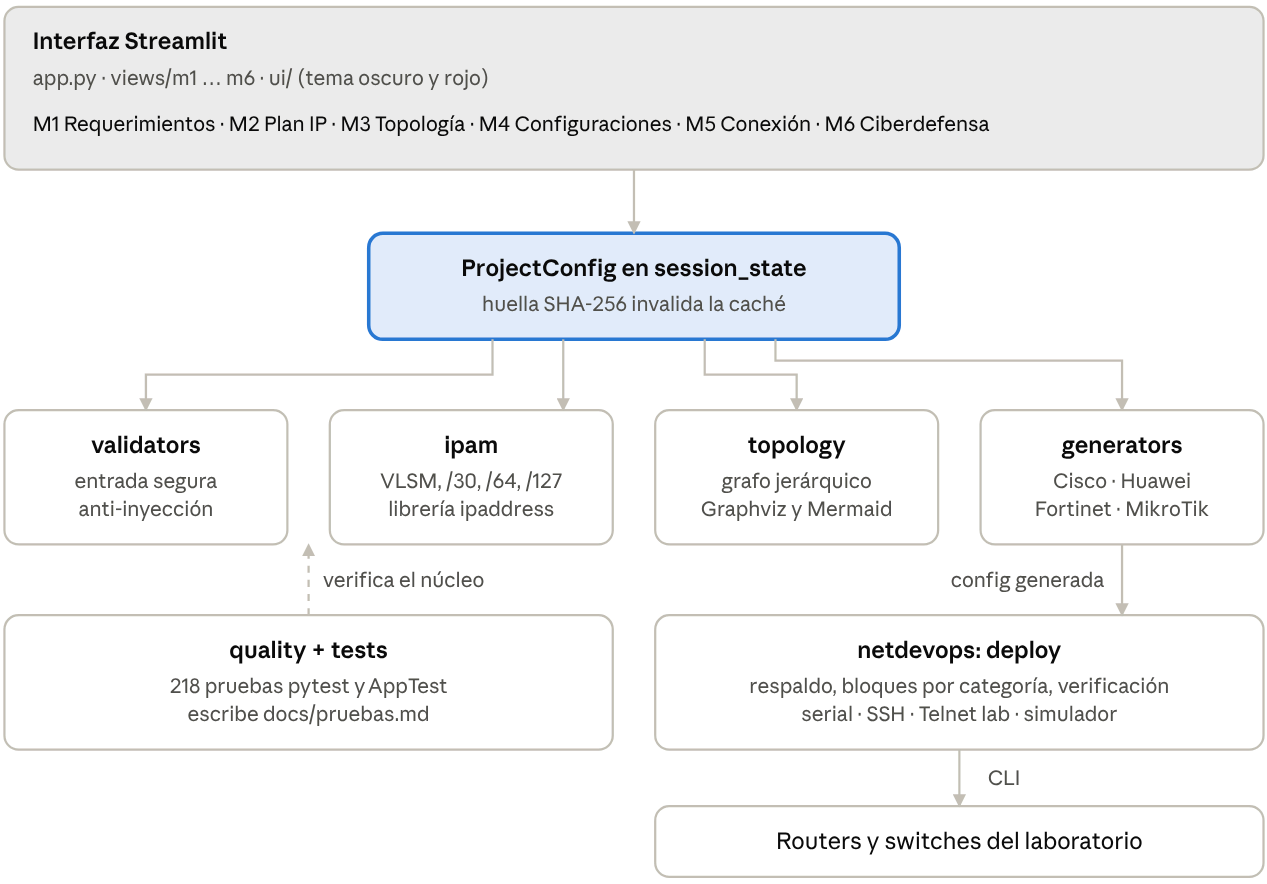
Redundancia de borde y núcleo (HSRP/VRRP, doble ISP) y despliegue masivo de todo el edificio en un solo paso.

3. Arquitectura de software

La aplicación separa un núcleo de lógica pura, que no depende de Streamlit, de la capa de interfaz. Por eso todo el cálculo se prueba con pytest sin abrir el navegador.

El objeto `ProjectConfig` vive en `st.session_state` y es la única fuente de verdad. Su huella SHA-256 invalida la caché: si cambia un parámetro en el Módulo 1, el plan IP, la topología, las configuraciones y la consola se recalculan solos.

Un solo estado global alimenta todos los módulos



Arquitectura de software · interfaz, estado global, núcleo y conexión a equipos

La interfaz solo lee y escribe el estado; el núcleo calcula todo a partir de él, y la configuración generada es la misma que se despliega en los equipos.

Estructura del proyecto

Carpeta / archivo	Responsabilidad
`app.py`	Punto de entrada, navegación y panel del diseño activo
`core/state.py`	`ProjectConfig` y estado global
`core/validators.py`	Validación estricta, anti-inyección, enmascarado de secretos
`core/ipam.py`	VLSM, enlaces /30, IPv6 /64 y /127
`core/topology.py`	Grafo jerárquico y exportación a Graphviz y Mermaid
`core/capacity.py`	Lista de materiales referencial y demanda de ancho de banda
`core/quality.py`	Auditoría de seguridad y porcentaje de confianza
`generators/`	Un motor de plantillas por fabricante
`netdevops/ssh_client.py`	SSH con paramiko y verificación de host
`netdevops/serial_client.py`	Cable de consola (COM) con pyserial
`netdevops/telnet_client.py`	Telnet restringido a laboratorios privados
`netdevops/deploy.py`	Despliegue guiado: bloques, respaldo, aplicar, verificar
`netdevops/simulator.py`	Equipos simulados con estado
`ui/` , `views/`	Sistema de diseño y una vista por módulo
`tests/`	218 pruebas (pytest y `streamlit.testing.AppTest`)
`docs/`	Arquitectura, plan IP, seguridad, pruebas, políticas de IA

Tecnologías

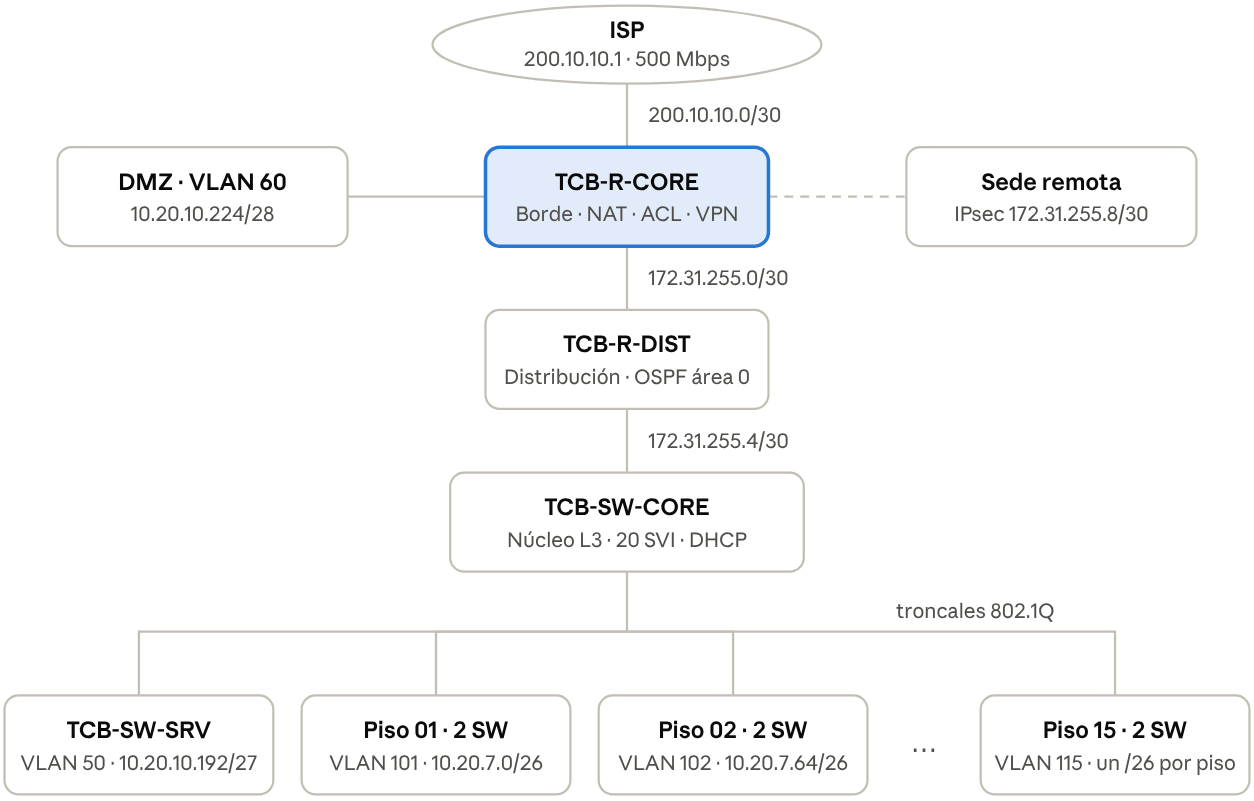
Tecnología	Uso
Python 3.11+	Lenguaje base
Streamlit 1.45+	Interfaz web
`ipaddress` (estándar)	Cálculo de subredes
paramiko 3.4+	SSH
pyserial 3.5+	Consola serial
Graphviz y Mermaid	Diagramas
python-dotenv	Carga de credenciales desde `.env`
pytest	Pruebas automatizadas

4. Arquitectura de red modelada

La red sigue el modelo jerárquico de tres capas más borde: ISP, router de borde, router de distribución, switch núcleo L3, switches de acceso por piso y una zona de servicios con servidores y DMZ.

Cada capa se une a la siguiente por un enlace /30 calculado

Escenario de referencia: 15 pisos, 600 hosts, red 10.20.0.0/16



Cada troncal de piso lleva también voz (20), CCTV (30), Wi-Fi (40) y gestión (99)

Gestión de todos los equipos en la VLAN 99 (10.20.6.128/25)

Red jerárquica del escenario de referencia · direcciones del plan calculado

Capa	Equipo	Función
Borde	R-CORE	NAT/PAT, ACL anti-spoofing, VPN, QoS de voz, DMZ
Distribución	R-DIST	Enrutamiento OSPF área 0
Núcleo	SW-CORE	Gateways (SVI) de todas las VLAN, DHCP, raíz STP
Acceso	SW-ACC-Pnn	Usuarios con voz, cámaras PoE, AP; port-security, BPDU guard, DHCP snooping
Servicios	SW-SRV, DMZ	Granja de servidores y zona desmilitarizada

Numeración de VLAN

VLAN	Uso	Regla de tamaño
100 + n	Datos del piso n	⌈hosts / pisos⌉ × (1 + crecimiento)
20	Voz IP	hosts × (1 + crecimiento)
30	CCTV	cámaras × pisos + 2 NVR
40	Wi-Fi clientes	60 % de los hosts
50	Servidores	/27
60	DMZ	/28 en el router de borde
99	Gestión	1 IP por equipo y AP + 10
999	Blackhole: nativa de troncales y puertos sin uso	—

Reglas de direccionamiento

1. **VLSM descendente:** los bloques se asignan de mayor a menor y alineados a su frontera natural, sin huecos.
2. **Gateway:** la primera IP útil de cada subred, como en Cisco Packet Tracer.
3. **Enlaces WAN /30:** 4 direcciones, 2 útiles; el lado superior toma la primera y el inferior la segunda.
4. **IPv6:** un /64 por VLAN con el número de VLAN como identificador (VLAN 101 → `2001:db8:acad:101::/64`) y /127 en enlaces punto a punto (RFC 6164).

El tamaño de cada subred se obtiene con la menor potencia de 2 que aloja los hosts más red y broadcast:

prefijo = 32 − ⌈$\log_2$(hosts + 2)⌉

Ejemplo, piso 01: 600 hosts / 15 pisos = 40; con 25 % de crecimiento, 50 hosts. Como 2^6 = 64 ≥ 52, el prefijo es /26 (62 útiles). Red 10.20.7.0, máscara 255.255.255.192, gateway 10.20.7.1, broadcast 10.20.7.63.

Enrutamiento

OSPFv2 y OSPFv3 en área 0 entre R-CORE, R-DIST y SW-CORE, con las SVI como interfaces pasivas. R-CORE inyecta la ruta por defecto y hace PAT hacia el ISP. Con MPLS, eBGP entre el CE (AS 65010) y el PE (AS 65000) anuncia la ruta sumarizada.

5. Módulos de la aplicación

La interfaz tiene un menú lateral con los seis módulos y un panel del diseño activo. Las capturas corresponden al escenario de referencia: Torre Corporativa Bogotá, 15 pisos, 600 hosts, fabricante Cisco, VPN IPsec.

5.1 Módulo 1: Requerimientos

El formulario recoge el escenario: nombre, pisos o zonas, hosts, crecimiento, presupuesto, ancho de banda del ISP, cámaras y AP por piso, tipo de VPN, servicios y redes base. Hay cuatro plantillas: edificio de 15 pisos, sede regional, campus de 8 bloques y sucursal de 1 piso.

Al guardar, el diseño se valida completo; si la red base no alcanza, se rechaza con un mensaje y el diseño vigente no cambia. Debajo aparecen la viabilidad (costo frente a presupuesto, demanda pico frente al enlace ISP) y la lista de materiales referencial.

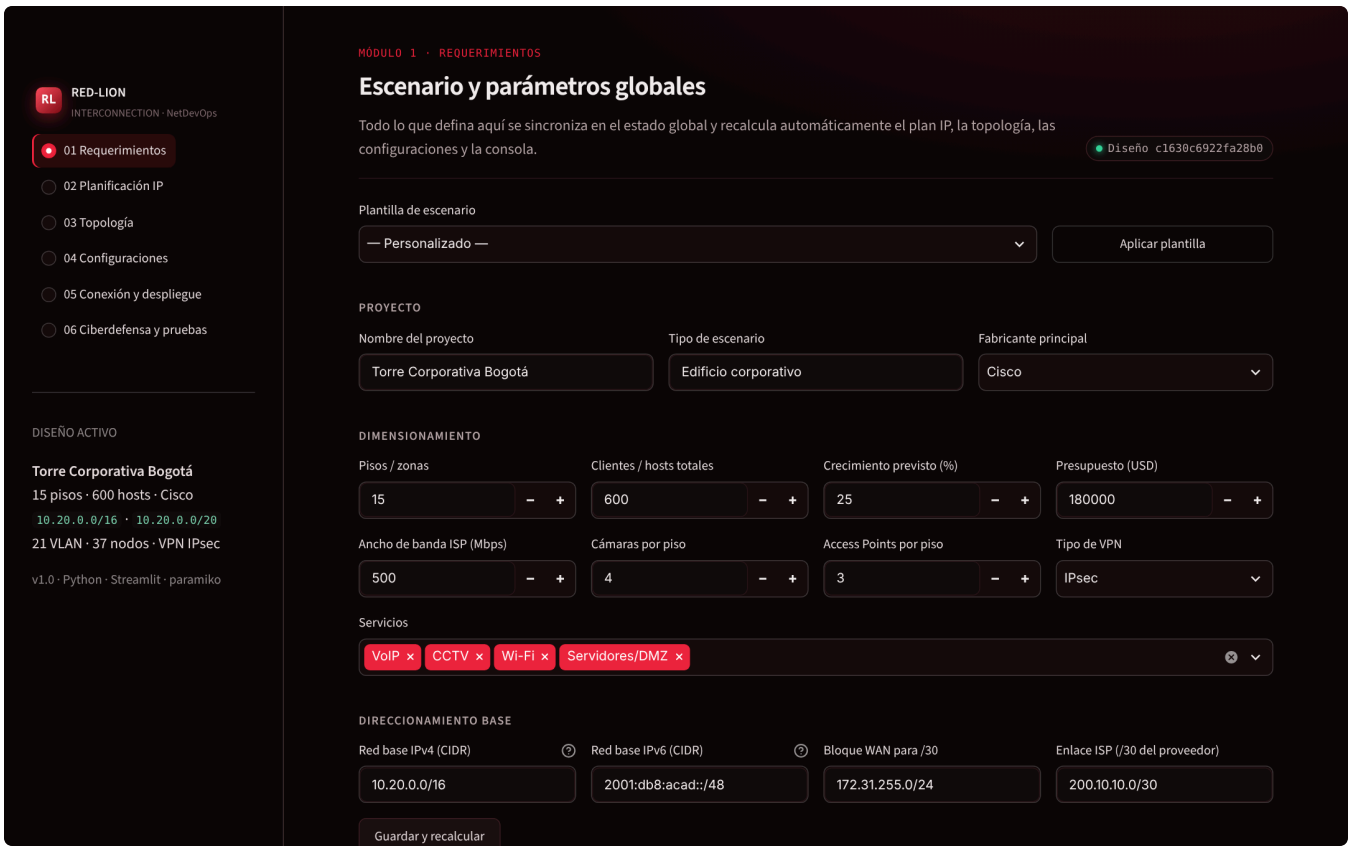


Figura 1. Formulario de requerimientos.

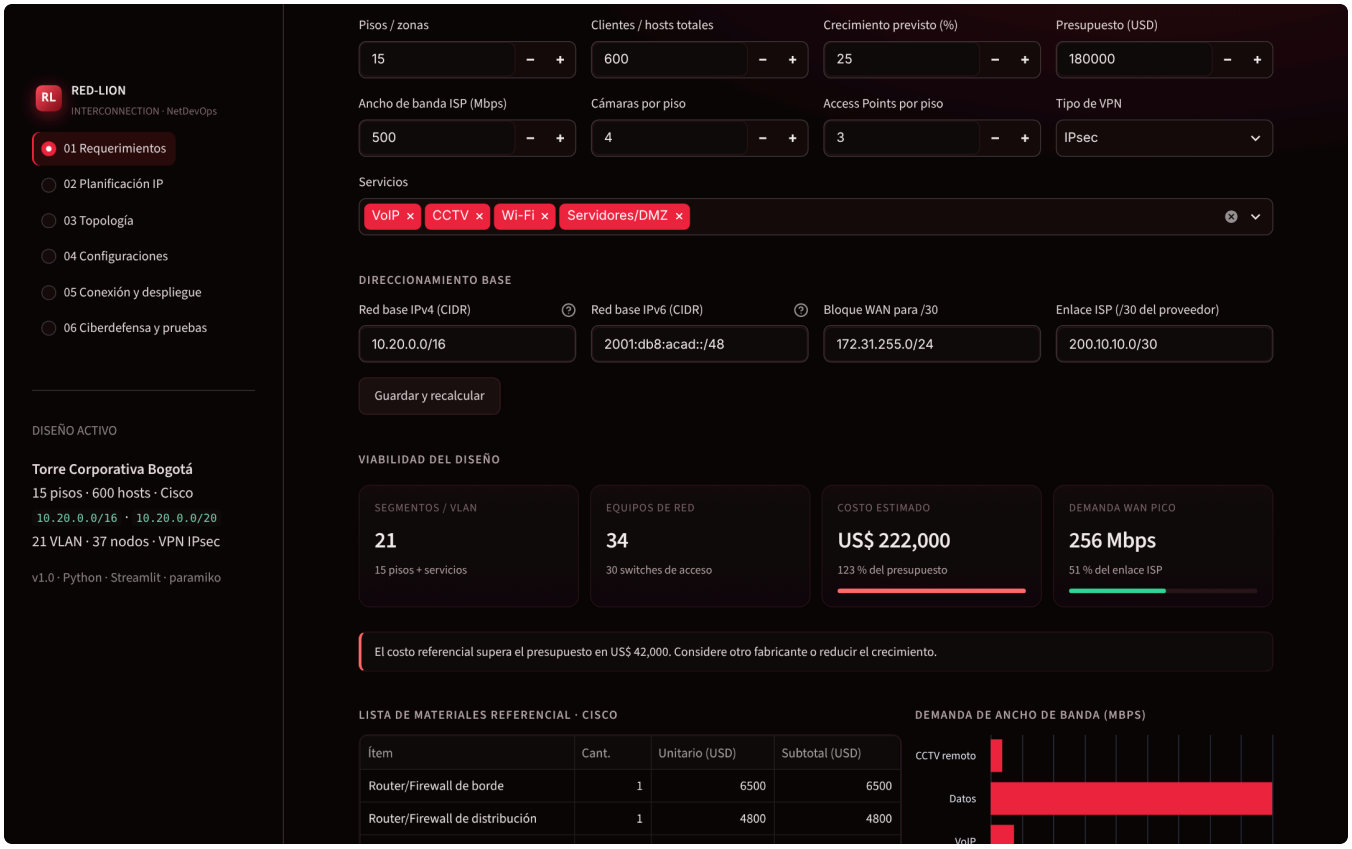
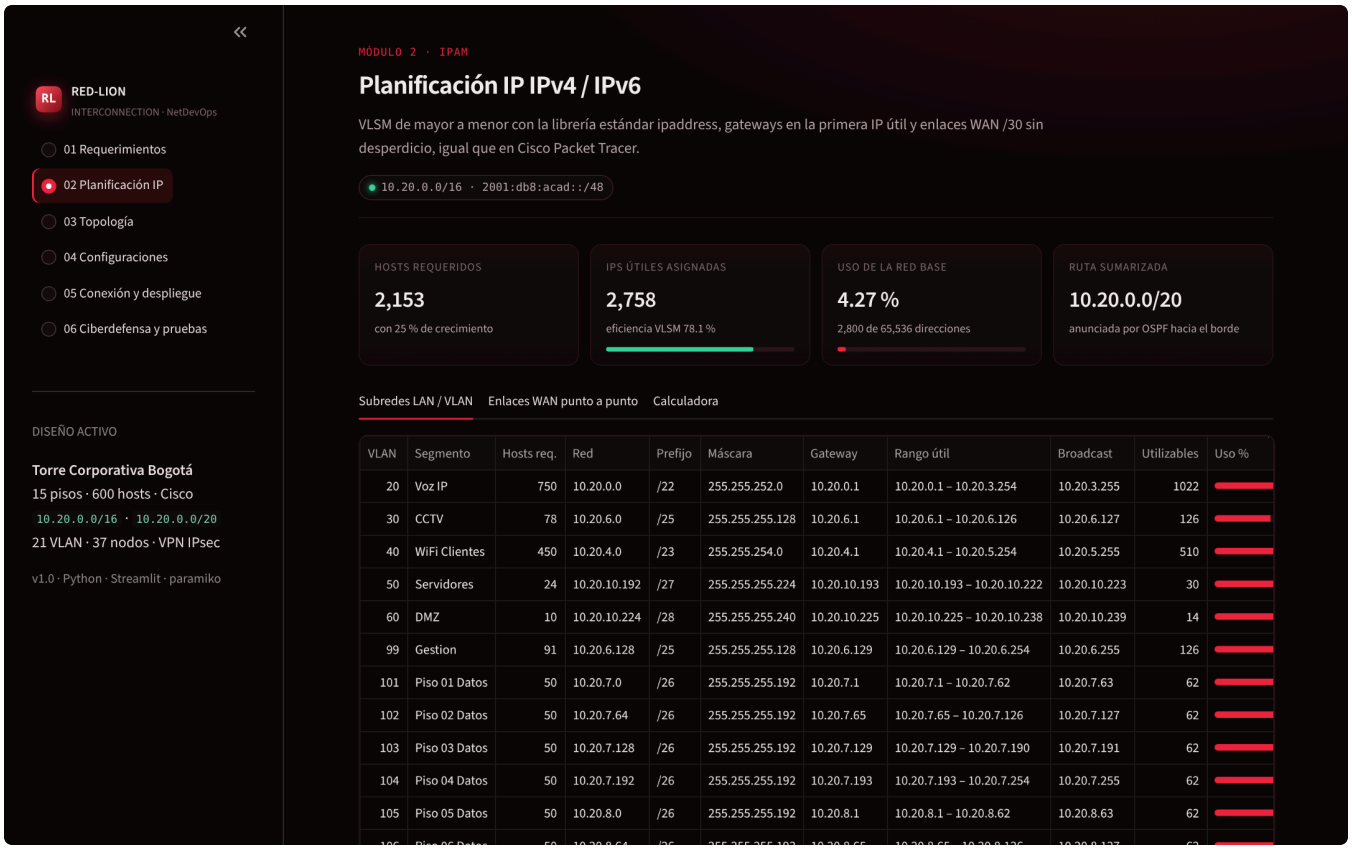


Figura 2. Viabilidad del diseño y lista de materiales.

En el escenario de referencia la app detecta que el costo referencial con Cisco (US$ 222.000) supera el presupuesto de US$ 180.000 en 23 %, y que la demanda pico de 256 Mbps usa el 51 % del enlace de 500 Mbps.

5.2 Módulo 2: Planificación IP

Muestra hosts requeridos, IP útiles asignadas, eficiencia VLSM, uso de la red base y la ruta sumarizada. La tabla por VLAN trae red, prefijo, máscara, gateway, rango útil, broadcast, porcentaje de uso y su /64 IPv6. Se exporta a CSV o Markdown e incluye una calculadora para verificar cualquier subred a mano.



VLAN	Segmento	Hosts req.	Red	Prefijo	Máscara	Gateway	Rango útil	Broadcast	Utilizables	Uso %
20	Voz IP	750	10.20.0.0	/22	255.255.252.0	10.20.0.1	10.20.0.1 – 10.20.3.254	10.20.3.255	1022	
30	CCTV	78	10.20.6.0	/25	255.255.255.128	10.20.6.1	10.20.6.1 – 10.20.6.126	10.20.6.127	126	
40	WiFi Clientes	450	10.20.4.0	/23	255.255.254.0	10.20.4.1	10.20.4.1 – 10.20.5.254	10.20.5.255	510	
50	Servidores	24	10.20.10.192	/27	255.255.255.224	10.20.10.193	10.20.10.193 – 10.20.10.222	10.20.10.223	30	
60	DMZ	10	10.20.10.224	/28	255.255.255.240	10.20.10.225	10.20.10.225 – 10.20.10.238	10.20.10.239	14	
99	Gestion	91	10.20.6.128	/25	255.255.255.128	10.20.6.129	10.20.6.129 – 10.20.6.254	10.20.6.255	126	
101	Piso 01 Datos	50	10.20.7.0	/26	255.255.255.192	10.20.7.1	10.20.7.1 – 10.20.7.62	10.20.7.63	62	
102	Piso 02 Datos	50	10.20.7.64	/26	255.255.255.192	10.20.7.65	10.20.7.65 – 10.20.7.126	10.20.7.127	62	
103	Piso 03 Datos	50	10.20.7.128	/26	255.255.255.192	10.20.7.129	10.20.7.129 – 10.20.7.190	10.20.7.191	62	
104	Piso 04 Datos	50	10.20.7.192	/26	255.255.255.192	10.20.7.193	10.20.7.193 – 10.20.7.254	10.20.7.255	62	
105	Piso 05 Datos	50	10.20.8.0	/26	255.255.255.192	10.20.8.1	10.20.8.1 – 10.20.8.62	10.20.8.63	62	

Figura 3. Plan de subredes LAN por VLAN.

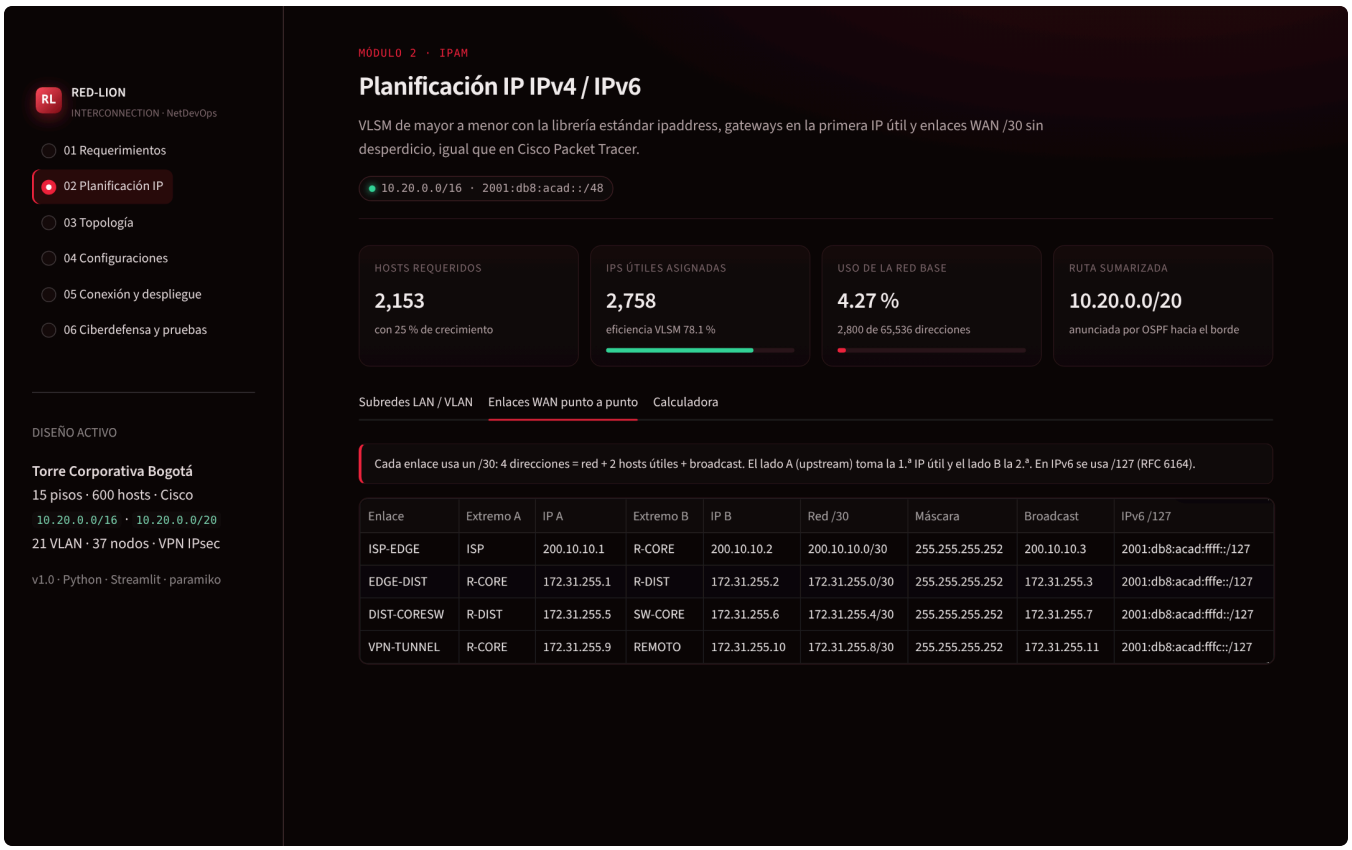


Figura 4. Enlaces WAN punto a punto /30 y /127.

5.3 Módulo 3: Topología

Dibuja la arquitectura jerárquica desde el plan, con la IP de gestión y las redes reales en cada nodo. Los switches de acceso se agrupan por piso para que un edificio grande siga siendo legible. Ofrece vista Graphviz, vista Mermaid interactiva, inventario de equipos y enlaces, y exportación a .dot, .mmd y .svg.

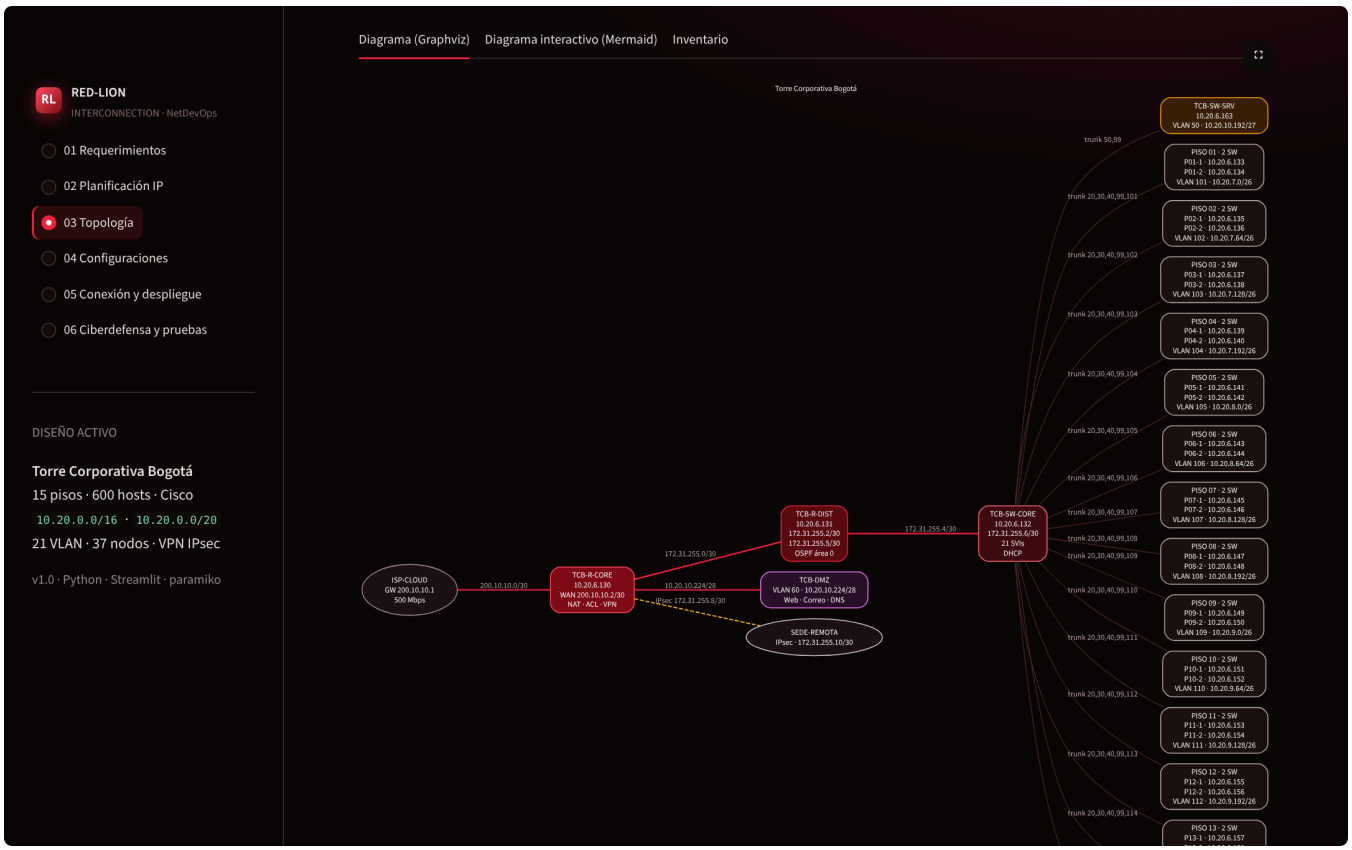


Figura 5. Topología jerárquica generada.

5.4 Módulo 4: Configuraciones

Genera un archivo .txt por equipo para el fabricante elegido: VLAN, interfaces, OSPFv2/v3, DHCP, NAT, QoS de voz, VPN y hardening. Las contraseñas nunca se escriben: quedan como marcadores `<<ADMIN_SECRET>>` o `<<VPN_PSK>>`. Se descarga un archivo o todos en un .zip.

Rol	Cisco	Huawei	Fortinet	MikroTik
Borde	ISR 4331	AR6120	FortiGate 200F	CCR2004-16G-2S+
Distribución	ISR 4331	AR6120	FortiGate 100F	CCR2004-16G-2S+
Núcleo	Catalyst 9300-48P	S6730-H48X6C	FortiSwitch 1048E	CRS518-16XS-2XQ
Acceso	Catalyst 9200-48P	S5735-L48P4X	FortiSwitch 448E-FPOE	CRS354-48P-4S+2Q+

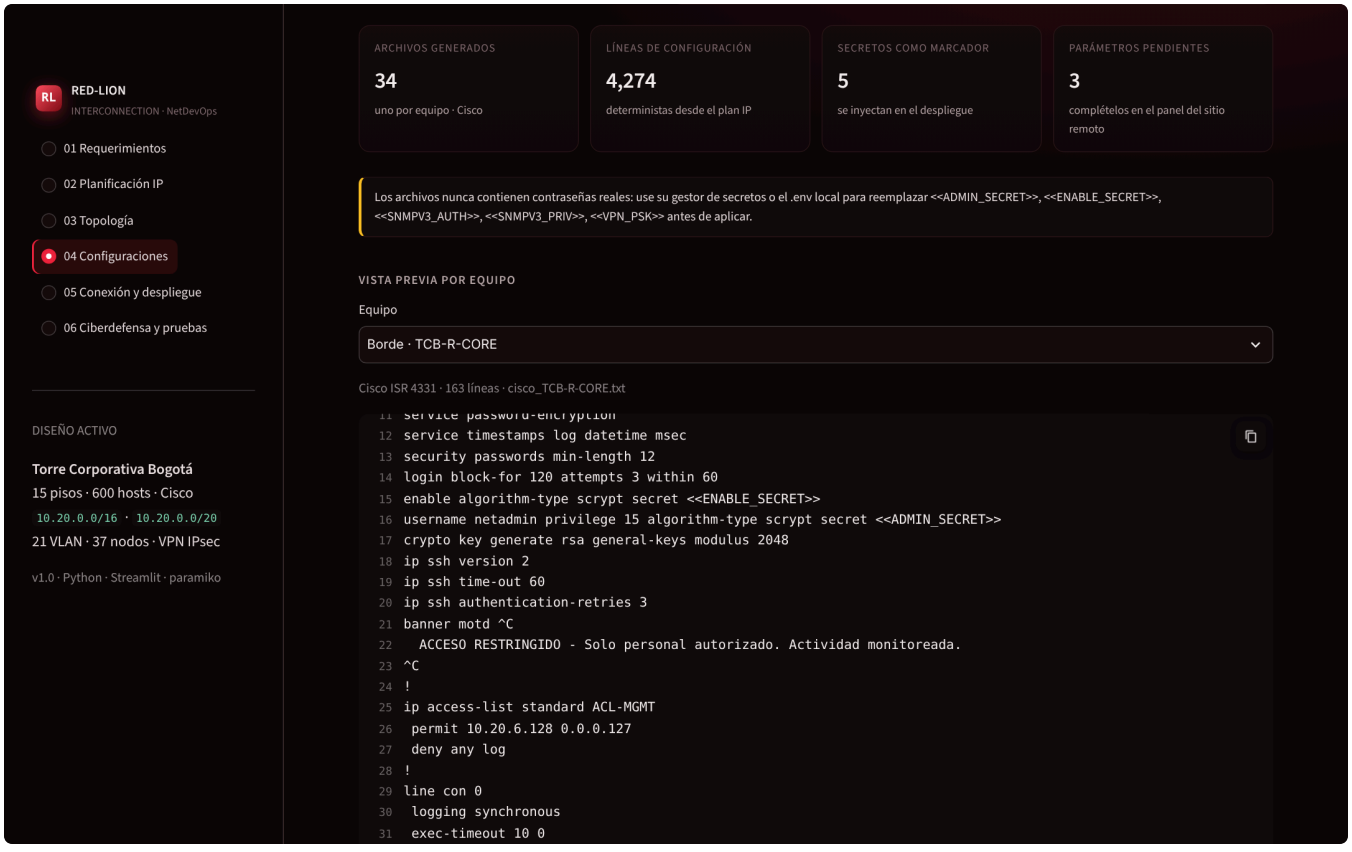


Figura 6. Vista previa de la configuración de un equipo.

5.5 Módulo 5: Conexión y despliegue

Se conecta a la terminal de los equipos en cuatro modos: simulación, consola serial, SSH y Telnet de laboratorio. La pestaña de despliegue guiado aplica la configuración y la terminal ejecuta comandos de consulta en lote. El flujo completo se detalla en la sección 6.

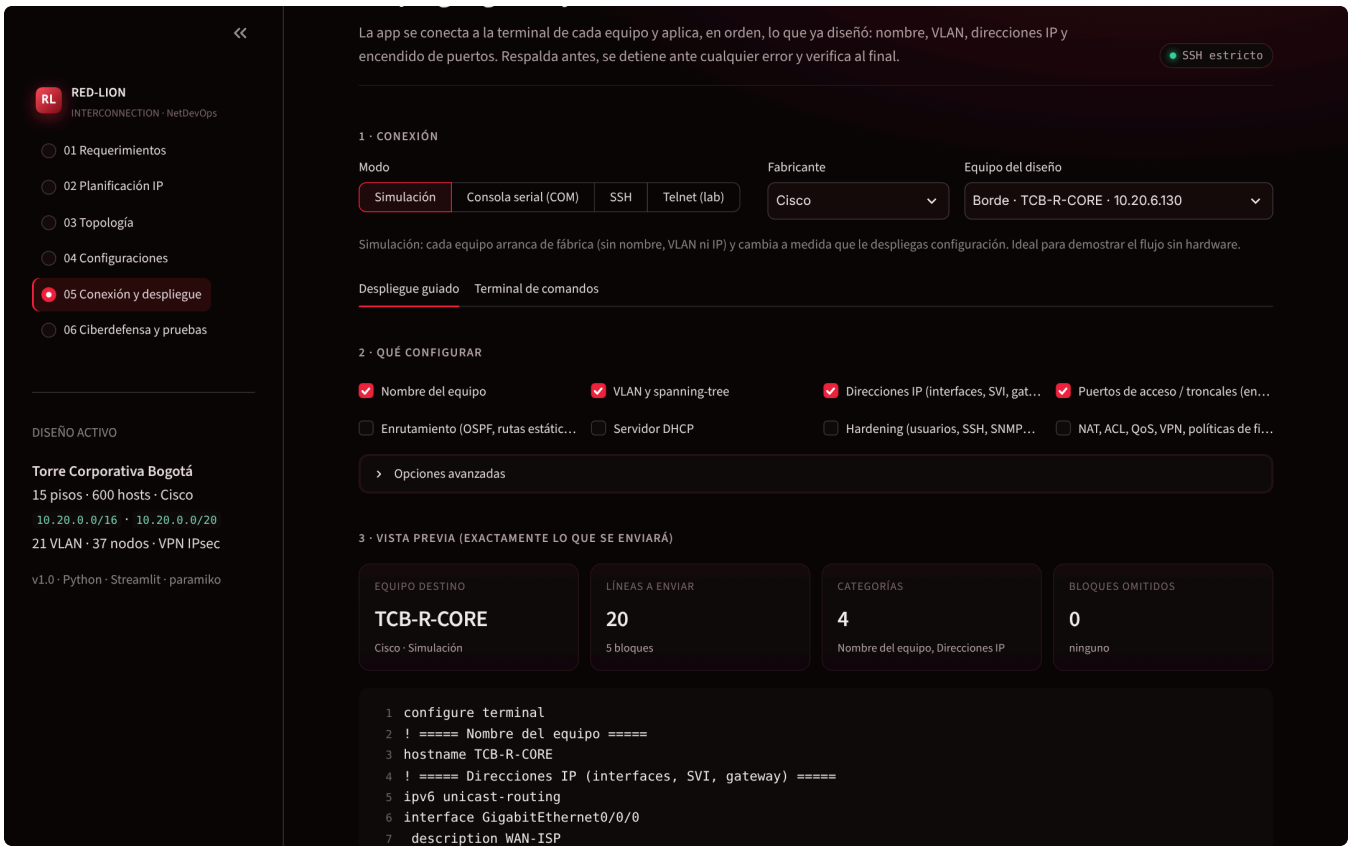


Figura 7. Panel de conexión y selección de categorías.

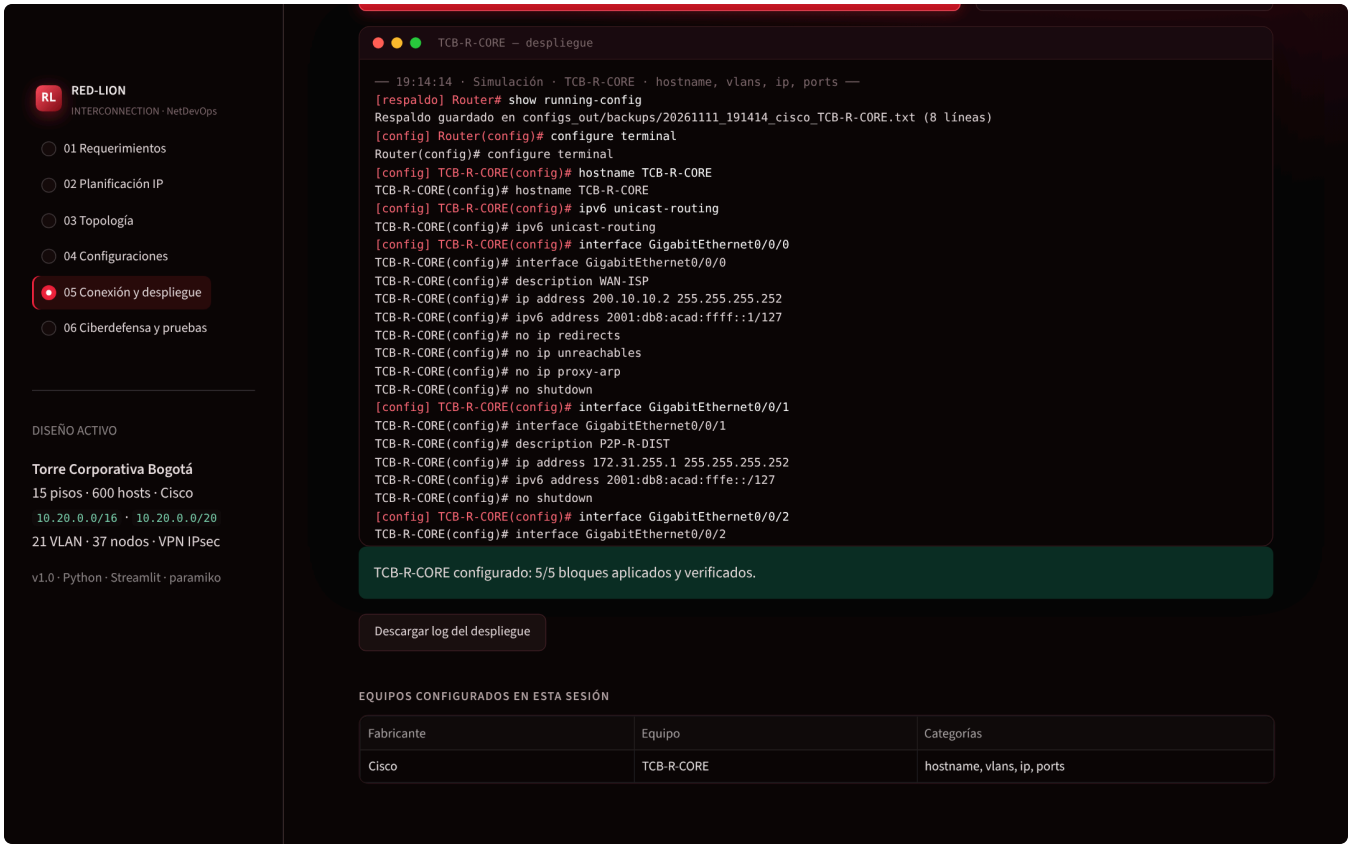


Figura 8. Despliegue simulado completado en TCB-R-CORE.

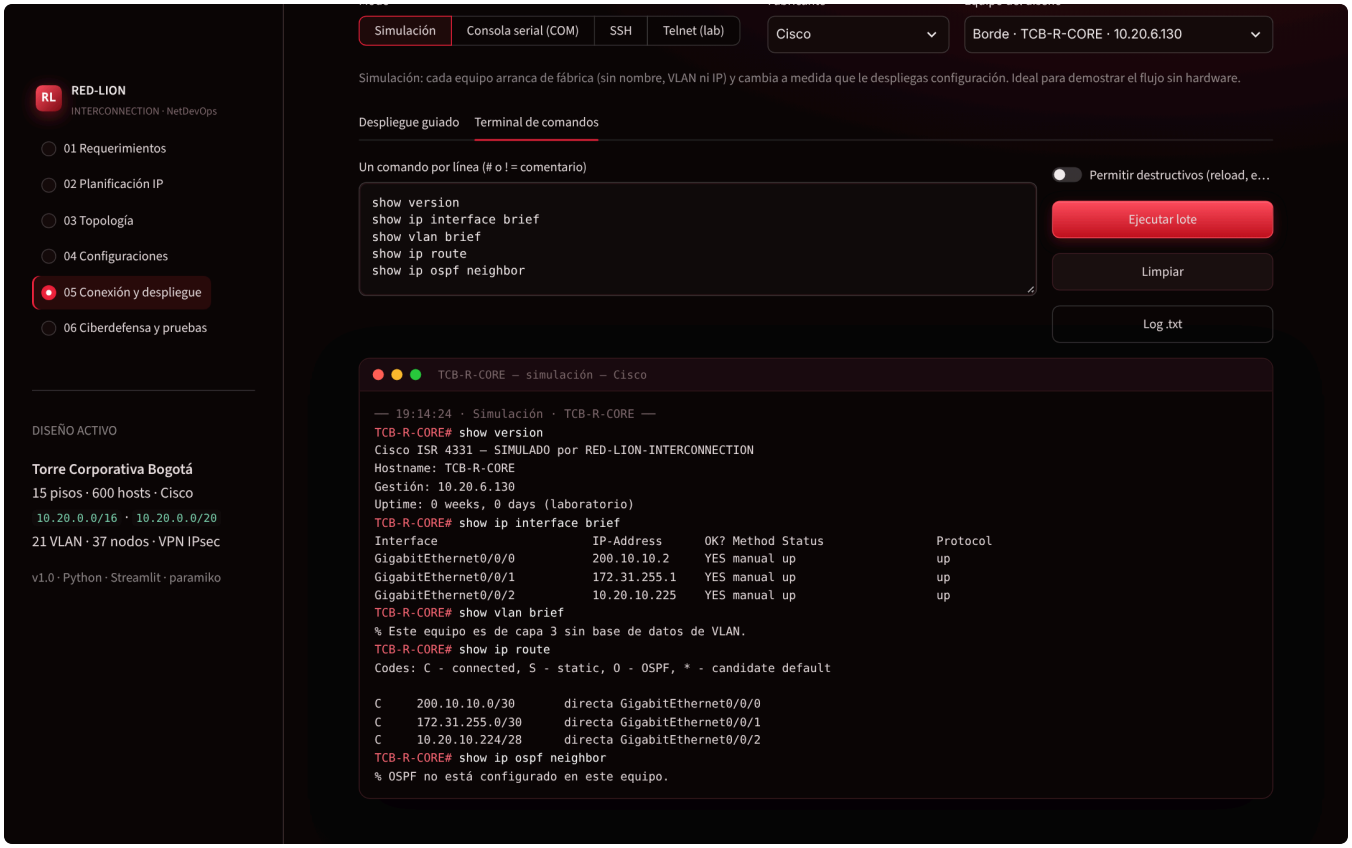


Figura 9. Terminal de comandos en simulación.

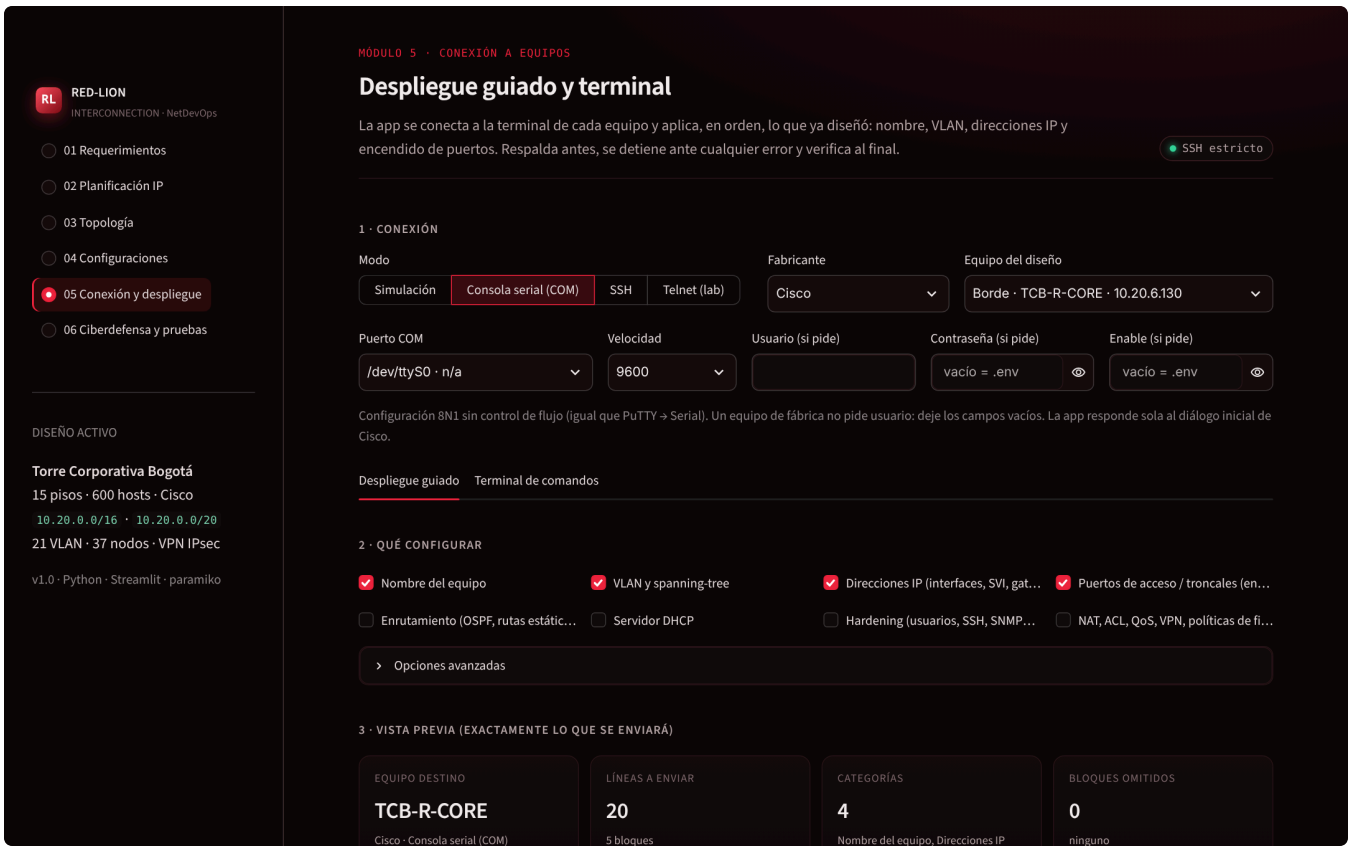


Figura 10. Modo consola serial (COM).

5.6 Módulo 6: Ciberdefensa y pruebas

Muestra el porcentaje de confianza, las pruebas superadas, los controles de seguridad y las categorías cubiertas. Un botón ejecuta la batería completa y escribe `docs/pruebas.md`. El laboratorio de validación deja probar el filtro anti-inyección y el enmascarado de secretos.

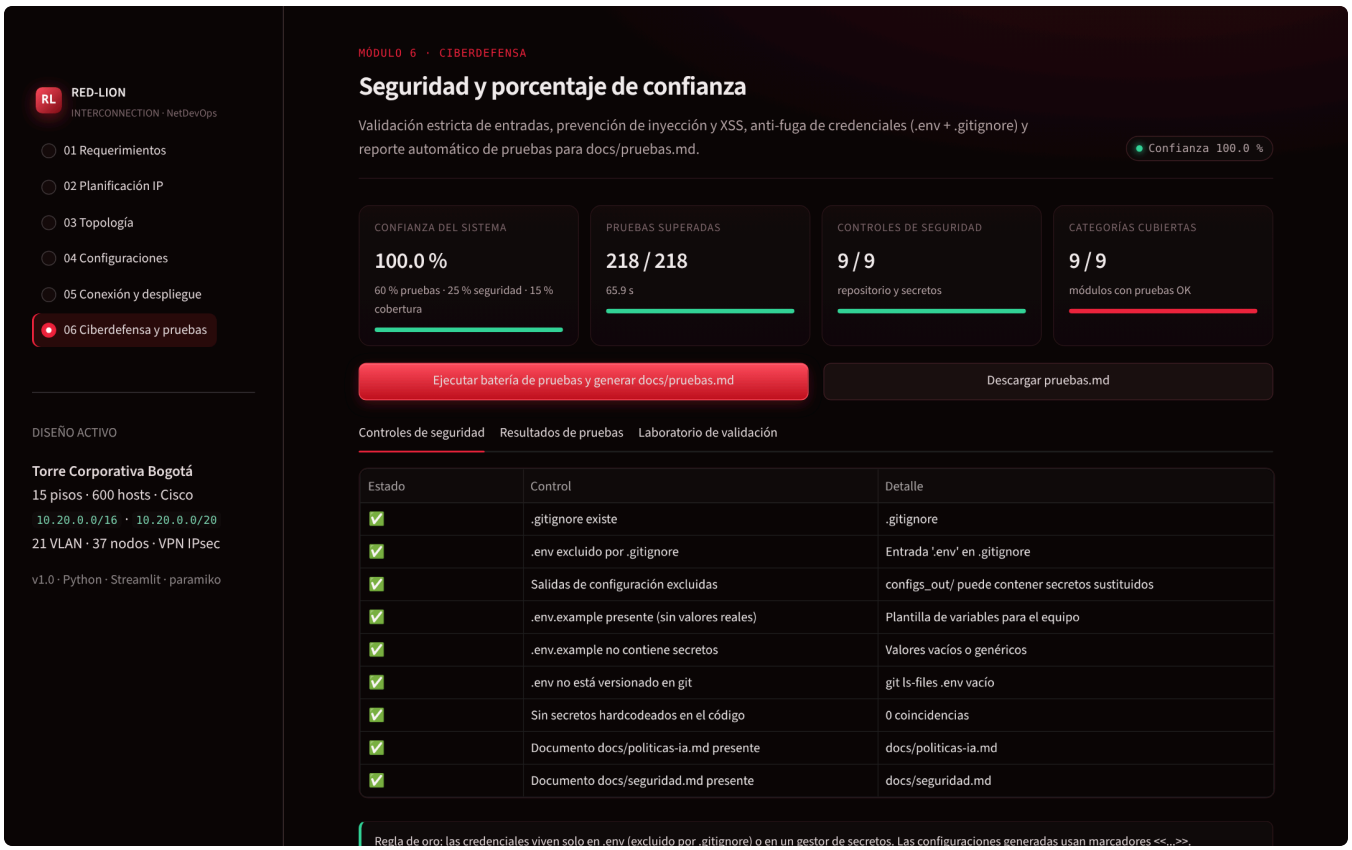


Figura 11. Panel de ciberdefensa con el porcentaje de confianza.

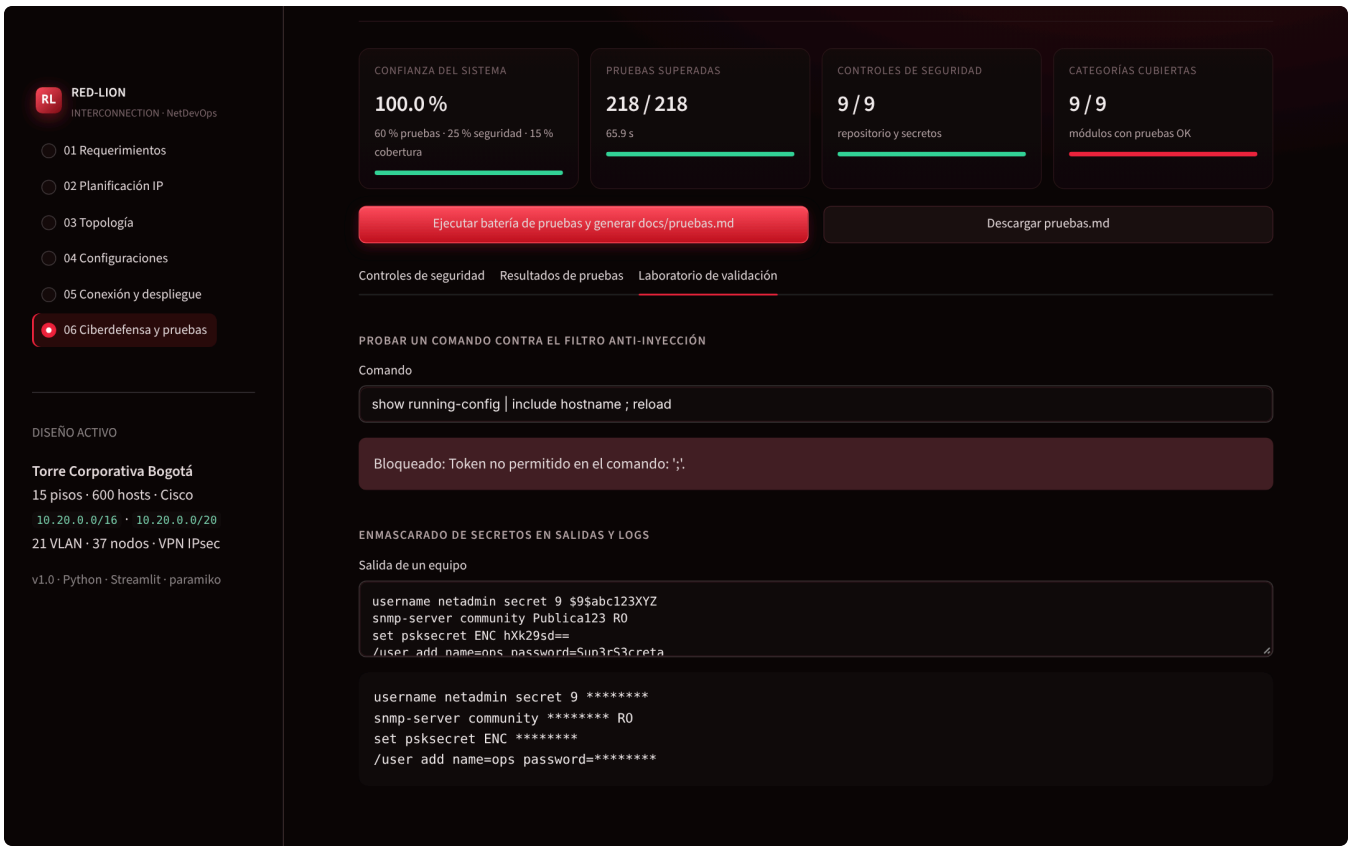


Figura 12. Filtro anti-inyección y enmascarado de secretos.

6. Conexión y despliegue a los equipos

El despliegue no escribe comandos propios: toma la misma configuración del Módulo 4, la parte en bloques y envía solo las categorías elegidas. Así lo que llega al equipo es exactamente lo que el plan calculó.

Modos de conexión

Modo	Cuándo se usa	Implementación
Consola serial (COM)	Equipo de fábrica, sin IP ni SSH; igual que PuTTY → Serial	pyserial, 9600 baudios 8N1; responde "no" al diálogo inicial de Cisco y entra a `enable`
SSH	Equipo con IP de gestión y SSH habilitado	paramiko, `RejectPolicy` contra `known_hosts`
Telnet (lab)	Emuladores GNS3, EVE-NG o eNSP	Socket propio; solo IP privadas y con `ENABLE_TELNET_LAB=true`
Simulación	Demostración sin hardware	Equipo de fábrica que cambia de estado al recibir cada categoría

Categorías que se pueden aplicar

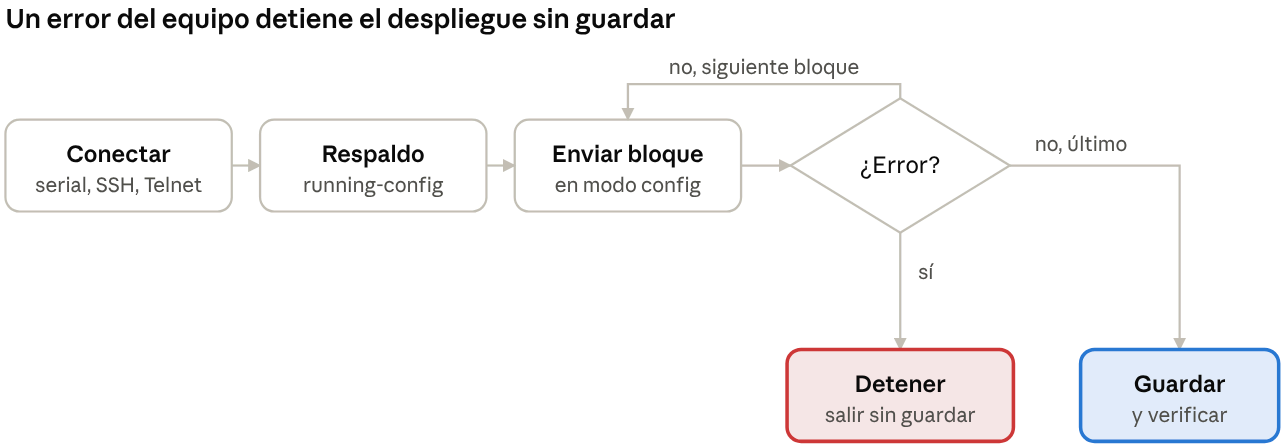
Categoría	Qué configura	Por defecto
Nombre del equipo	`hostname` / `sysname` / `set hostname` / `/system identity`	Sí
VLAN y spanning-tree	Creación de VLAN, RSTP, raíz	Sí
Direcciones IP	Interfaces, SVI, gateway de gestión	Sí
Puertos	Acceso y troncales encendidos con su VLAN; los no usados, apagados	Sí
Enrutamiento	OSPFv2/v3 y rutas estáticas	No
DHCP	Pools por VLAN	No

Categoría	Qué configura	Por defecto
Hardening	Usuario local, SSH v2, SNMPv3, banner	No
Servicios	NAT, ACL, QoS, VPN, políticas	No

Las dependencias se agregan solas: en un switch, IP y puertos arrastran las VLAN; enrutamiento, DHCP y servicios arrastran IP. Si no se aplican servicios, se quitan de las interfaces líneas como `ip nat` o `ip access-group`. Un bloque con un marcador sin valor (por ejemplo, la IP pública del peer VPN) se omite completo.

Flujo de ejecución

1. **Respaldo:** `show running-config` (o su equivalente) se guarda en `configs_out/backups/`.
2. **Modo configuración:** `configure terminal` en Cisco o `system-view` en Huawei.
3. **Bloque por bloque:** cada respuesta se analiza con patrones de error del fabricante; ante un error se detiene y sale sin guardar.
4. **Guardar:** `write memory` o `save`, opcional.
5. **Verificar:** `show ip interface brief`, `show vlan brief`, `show ip route` u otros según el rol.



Flujo del despliegue guiado · respaldo, bloques, detención ante error, guardar

En equipos reales hay que escribir el nombre del equipo para confirmar, y el hardening solo se habilita si `ADMIN_SECRET` (y en Cisco `ENABLE_SECRET`) están en el `.env`, para no dejar el equipo sin acceso.

7. Ciberdefensa

Cada punto de entrada de la aplicación tiene un control específico; ninguna credencial sale del `.env` local ni llega al repositorio.

Activo	Amenaza	Control implementado
Formularios (M1)	Valores fuera de rango, CIDR mal formados, nombres que rompen el CLI	Lista blanca, `ipaddress(strict=True)`, el plan se recalcula antes de guardar
Terminal (M5)	Inyección de comandos: `;`, `&&`, `$()`, saltos de línea ocultos	Tokens prohibidos, caracteres de control, máx. 256 caracteres y 50 comandos
Terminal (M5)	`reload`, `erase`, `factoryreset` por accidente	Detección por fabricante; exige activar una confirmación
Despliegue (M5)	Cambio en el equipo equivocado o configuración a medias	Confirmar escribiendo el hostname, respaldo, detención sin guardar
Interfaz web	XSS desde la salida de un equipo	`html.escape` antes de pintar; Mermaid en modo `strict`
SSH	Suplantación del equipo (MITM)	`RejectPolicy` contra `known_hosts`; modo laboratorio solo con bandera explícita
Telnet	Credenciales en claro	Solo IP privadas o loopback y con bandera en `.env`
Credenciales	Fuga a git, logs o pantalla	`.env` en `.gitignore`, contraseña oculta en `repr`, enmascarado de salidas
Configuraciones generadas	Secretos en archivos descargables	Marcadores `<<...>>` que se reemplazan solo al enviar
Streamlit	CSRF y fuga de trazas	`enableXsrfProtection` activo, `showErrorDetails` desactivado

Hardening incluido en las configuraciones

- SSH v2, Telnet deshabilitado y acceso de gestión limitado a la VLAN 99.
- Contraseñas con hash fuerte: `scrypt` en Cisco, `irreversible-cipher` en Huawei.
- Bloqueo tras intentos fallidos y SNMPv3 con autenticación y cifrado.
- En acceso: port-security, BPDU guard, DHCP snooping, VLAN nativa 999 y puertos sin uso apagados.
- En el borde: ACL anti-spoofing RFC 1918, PAT e IPsec IKEv2 con AES-256-GCM.

8. Pruebas y porcentaje de confianza

Las 218 pruebas automatizadas pasan y los 9 controles de seguridad se cumplen, lo que da un porcentaje de confianza del 100 %. El reporte se genera ejecutando pytest; no se edita a mano.

Fórmula

Confianza = 0,60 × (pruebas OK / total) + 0,25 × (controles OK / controles) + 0,15 × (categorías OK / categorías)

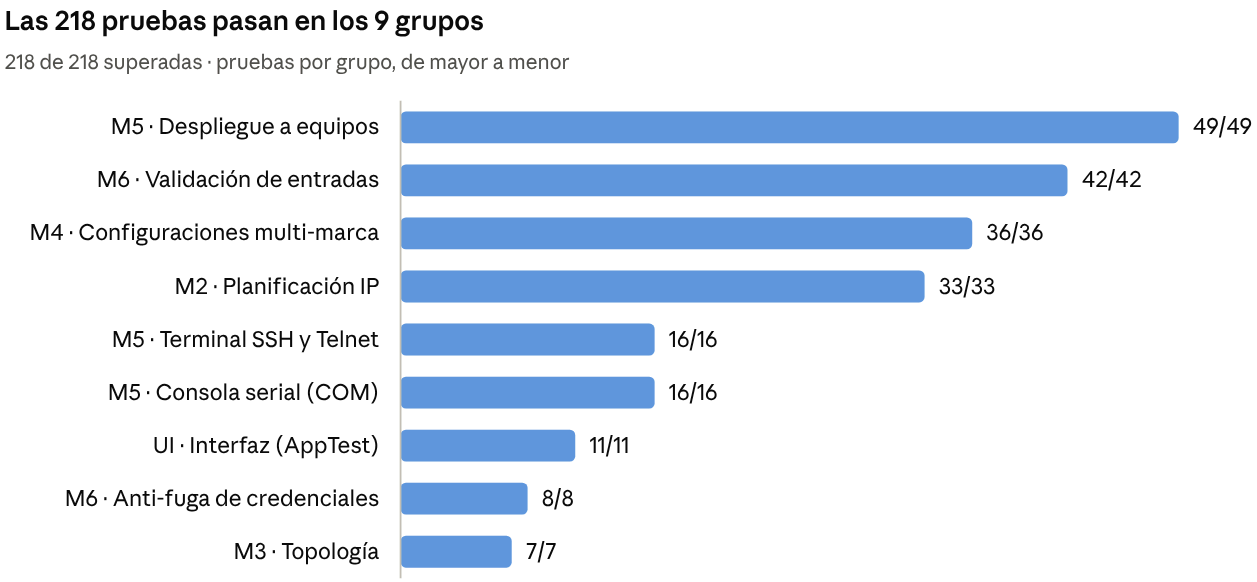
Componente	Peso	Resultado	Aporte
Pruebas superadas	60 %	218 / 218	60,0
Controles de seguridad	25 %	9 / 9	25,0
Cobertura de categorías	15 %	9 / 9	15,0
Total			**100,0 %**

Qué se prueba

- **Unitarias:** subredes contra cálculos manuales (red, broadcast, máscara, gateway) y regla de 2 IP útiles en /30.
- **Propiedades:** ninguna subred se solapa, todas caben en la red base, cada VLAN tiene un /64 único, escenarios de 1 a 40 pisos.
- **Generadores:** un archivo por equipo, sin secretos reales, con las IP del plan, para los 4 fabricantes y las 4 VPN.

- **Integración SSH:** un servidor SSH local emula un Cisco (modo usuario → `enable` → configuración) y valida el despliegue y la detención ante errores.
- **Consola serial:** un Cisco de fábrica emulado con diálogo inicial y login, conectado con la misma API de pyserial.
- **Seguridad:** inyección de comandos, XSS, enmascarado de contraseñas y escáner de secretos.
- **Interfaz:** cada módulo se ejecuta con `streamlit.testing.AppTest` sin excepciones.

Pruebas por módulo



docs/pruebas.md · ejecución del 11 de noviembre de 2026

El despliegue a equipos concentra más pruebas porque se valida con los 4 fabricantes, los 5 tipos de equipo y contra un servidor SSH emulado.

9. Instalación, ejecución y repositorio

El código está publicado en github.com/angelmacruz123/red-lion-interconnection. En Windows basta con descargarlo y hacer doble clic en `INICIAR.bat`.

Opción A: Windows con doble clic

1. Instalar Python 3.11 o superior desde python.org, marcando "Add Python to PATH".
2. En GitHub: **Code → Download ZIP** y descomprimir.

3. Doble clic en `INICIAR.bat`: crea el entorno, instala dependencias, crea el `.env` y abre `http://localhost:8501`.

Opción B: línea de comandos

```
git clone https://github.com/angelmacruz123/red-lion-interconnection.git
cd red-lion-interconnection
python -m venv .venv
.venv\Scripts\activate          # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
copy .env.example .env          # macOS/Linux: cp .env.example .env
streamlit run app.py
```

Verificar las pruebas

```
pytest -q
python scripts/generate_test_report.py --min 90
```

Variables del `.env`

Variable	Uso
`NET_USERNAME`, `NET_PASSWORD`	Credenciales por defecto para conectarse a los equipos
`ENABLE_SECRET`	Contraseña de `enable` en Cisco
`ADMIN_SECRET`, `VPN_PSK`, `SNMPV3_AUTH`, `SNMPV3_PRIV`	Secretos que reemplazan los marcadores al desplegar
`ALLOW_UNKNOWN_HOSTS`	`true` solo en laboratorio para aceptar claves SSH nuevas
`ENABLE_TELNET_LAB`	`true` para permitir Telnet hacia IP privadas

Entregables por corte

Corte	Evidencia en el repositorio
2	Módulos 1 a 4, `core/`, `generators/`, `docs/arquitectura.md`, `docs/plan-direccionamiento.md`
3	Módulos 5 y 6, `netdevops/`, `tests/`, `docs/pruebas.md`, `docs/seguridad.md`, `docs/politicas-ia.md`

10. Limitaciones, trabajo futuro y conclusiones

Limitaciones

- Un solo router de borde y un solo núcleo: no se modela redundancia (HSRP/VRRP ni doble ISP).
- El despliegue se hace equipo por equipo, no del edificio completo en un paso.
- La conexión real se validó contra equipos Cisco emulados (SSH y consola); falta la prueba en hardware físico del laboratorio.
- Las sintaxis de FortiSwitch y MikroTik varían entre versiones; deben probarse en GNS3, EVE-NG o eNSP antes de producción.
- Los precios de la lista de materiales son referenciales.
- Si la app se publica en la nube, la consola serial y el SSH a un laboratorio local no están disponibles; requieren ejecutar la app en un PC conectado a los equipos.

Trabajo futuro

1. Redundancia de primer salto y doble enlace ISP.
2. Despliegue en lote de todo el edificio con orden por capas y reversión automática desde el respaldo.
3. Lectura de la configuración actual del equipo para mostrar solo las diferencias antes de aplicar.
4. Integración con monitoreo (por ejemplo, Zabbix) usando la VLAN de gestión y SNMPv3 ya configurados.

Conclusiones

- Usar los requerimientos como única fuente de verdad elimina inconsistencias: el plan IP, el diagrama, las configuraciones y el despliegue siempre coinciden.
- El VLSM descendente con `ipaddress` asigna 21 subredes y 3 enlaces /30 sin solapamientos ni huecos, verificado con pruebas de propiedades.
- Separar el núcleo de la interfaz permitió 218 pruebas automáticas que miden la calidad con un indicador objetivo.
- La conexión por consola serial cierra el ciclo: un equipo de fábrica puede quedar configurado desde la app sin escribir comandos a mano.
- Las salvaguardas (respaldo, detención ante error, confirmación y secretos fuera del código) hacen que la automatización sea segura en un entorno real.

Referencias

Cruz Gonzalez, A. M. (2026). *RED-LION-INTERCONNECTION* [Código fuente]. GitHub. https://github.com/angelmacruz123/red-lion-interconnection

Kohno, M., Nitzan, B., Bush, R., Matsuzaki, Y., Colitti, L., & Narten, T. (2011). *Using 127-bit IPv6 prefixes on inter-router links* (RFC 6164). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc6164

Liechti, C. (s. f.). *pySerial documentation*. https://pyserial.readthedocs.io

Moy, J. (1998). *OSPF version 2* (RFC 2328). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc2328

Oppenheimer, P. (2011). *Top-down network design* (3.ª ed.). Cisco Press.

OWASP Foundation. (2021). *OWASP Top 10*. https://owasp.org/Top10/

Paramiko. (s. f.). *Paramiko documentation*. https://www.paramiko.org

Python Software Foundation. (s. f.). *ipaddress: IPv4/IPv6 manipulation library*. https://docs.python.org/3/library/ipaddress.html

Rekhter, Y., Moskowitz, B., Karrenberg, D., de Groot, G. J., & Lear, E. (1996). *Address allocation for private internets* (RFC 1918). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc1918

Streamlit. (s. f.). *Streamlit documentation*. https://docs.streamlit.io